Characterizing Speculative Decoding Dynamics for Large Language Models on Consumer-Class GPUs

Anonymous Submission
Author details omitted for double-blind review

***Abstract*—Speculative decoding is often reported to reduce large language model (LLM) latency, but behavior on consumer GPUs remains highly hardware- and runtime-dependent. We present a controlled study across two model families (Qwen2.5 and Qwen3) and two consumer setups (RTX4090 and RTX5090-laptop) under fixed policies ($k \in \{4, 8, 16\}$; deterministic and stochastic). On RTX4090 (Qwen3), the best setting is deterministic $k{=}4$ ($S{=}2.8548$), and speedup degrades monotonically as k increases. Deterministic decoding outperforms stochastic decoding at every matched k. On RTX5090-laptop, the same ordering trends hold (low-k best; deterministic $>$ stochastic), but absolute speedups remain below 1 under the tested stack, showing limited portability of absolute gains. Sample-level variance, time-to-first-token (TTFT), time-per-output-token (TPOT), and Holm-Bonferroni corrected paired tests with effect sizes are reported for statistical transparency. Qwen2.5 results show the same qualitative low-k and deterministic-preference trends, supporting cross-family consistency at the trend level. Overall, this work provides reproducible, deployment-oriented characterization and policy guidance rather than a universally throughput-improving algorithm.** **Speculative decoding is often reported to reduce large language model (LLM) latency, but behavior on consumer GPUs remains highly hardware- and runtime-dependent. We present a controlled study across two model families (Qwen2.5 and Qwen3) and two consumer setups (RTX4090 and RTX5090-laptop) under fixed policies ($k \in \{4, 8, 16\}$; deterministic and stochastic). On RTX4090 (Qwen3), the best setting is deterministic $k{=}4$ ($S{=}2.8548$), and speedup degrades monotonically as k increases. Deterministic decoding outperforms stochastic decoding at every matched k. On RTX5090-laptop, the same ordering trends hold (low-k best; deterministic $>$ stochastic), but absolute speedups remain below 1 under the tested stack, showing limited portability of absolute gains. Sample-level variance, time-to-first-token (TTFT), time-per-output-token (TPOT), and Holm-Bonferroni corrected paired tests with effect sizes are reported for statistical transparency. Qwen2.5 results show the same qualitative low-k and deterministic-preference trends, supporting cross-family consistency at the trend level. Overall, this work provides reproducible, deployment-oriented characterization and policy guidance rather than a universally throughput-improving algorithm.**

***Index Terms*—speculative decoding, large language models, inference acceleration, benchmarking, reproducibility**

I. Introduction

Autoregressive decoding in large language models (LLMs) is inherently sequential: each generated token requires a full forward pass of the target model. This constraint directly limits single-stream throughput in interactive inference settings. Speculative decoding mitigates this bottleneck by adding a smaller draft model that proposes a length-k token block, followed by one target-side verification step; modified rejection sampling preserves the target distribution [1], [2].

Although speculative decoding is now well studied, deployment guidance remains uncertain for consumer-GPU settings. Reported gains in the literature are often obtained on data-center hardware and are sensitive to model pairing, proposal length, and sampling regime. For practitioners operating on a single RTX-class GPU, the key questions are practical: which draft size is worthwhile, which k is optimal, and whether deterministic versus stochastic decoding changes the speed–quality frontier.

This study addresses three practical gaps for deployment-facing evidence: (i) broader hardware/model validation, (ii) clearer sample-level variance and statistical testing details, and (iii) explicit positioning of the adaptive-k run-set policy as a robustness mechanism rather than a speedup-maximizing algorithm.

This direction matters for privacy-sensitive, latency-critical local use, including local workstations and edge-side decision-support workflows. We therefore position this paper as a hardware-aware characterization study, rather than a new decoding algorithm.

Within this scope, the adaptive-k run set is not introduced to exceed the peak throughput of the best static speculative setting. Instead, we use an adaptive-k policy as a hardware-aware tail-latency guardrail for difficult inputs, trading some mean throughput for more controlled runtime behavior under acceptance drops.

A. Research questions

We address four research questions:

RQ1. How much end-to-end speedup does vanilla speculative decoding deliver against autoregressive decoding on consumer RTX hardware with Qwen2.5 and Qwen3 family pairings, and how close is this to a practical hardware-constrained upper bound?

RQ2. How do draft model size and draft length jointly determine token acceptance and net latency, and what do these trends imply about memory-traffic and kernel-launch overhead on consumer GPUs?

RQ3. How sensitive is the speed–quality trade-off to the sampling regime (deterministic vs. stochastic) and to task entropy?

RQ4. Do the ranking trends (optimal k, regime preference) transfer across consumer-GPU profiles (desktop RTX4090 vs. RTX5090-laptop), and where do absolute gains fail to transfer?

B. Contributions

This paper makes the following contributions:

1) A reproducible consumer-GPU protocol with frozen sample manifests, matched prompts, and two decoding regimes across GSM8K, MMLU subset, and CNN/DailyMail.
2) A two-family fixed-policy study combining Qwen2.5 and Qwen3 configurations, with $k \in \{4, 8, 16\}$ and deterministic/stochastic regimes, reported with latency, TTFT, TPOT, acceptance, and quality metrics.
3) Cross-hardware validation across desktop RTX4090 and RTX5090-laptop configurations, showing trend portability (best at low k, deterministic preference) but non-portable absolute gains.
4) Expanded statistical reporting: sample-level mean±std summaries, one-sided paired t-tests with Holm-Bonferroni correction, and Cohen's d effect sizes for each fixed-k configuration against its regime-matched baseline.
5) An adaptive-k run-set interpretation that prioritizes runtime stability and tail behavior over peak throughput claims.

The rest of the paper is organized as follows. Section II reviews related work. Section III defines the experimental protocol; Section IV defines metrics. Section V describes implementation and reproducibility controls. Section VI reports the main empirical findings, including brief adaptive-k evidence. Section VII discusses implications, limitations, and conclusions.

II. Related Work

Prior work on accelerating LLM inference falls into four broad categories relevant to this study: (i) vanilla speculative decoding with a separate draft model, (ii) architectural modifications that reduce or eliminate drafting cost, (iii) diffusion-based parallel drafters, and (iv) cascaded or precision-oriented systems techniques. We review each in turn before positioning this paper's adaptive-k contribution relative to them.

a) Vanilla speculative decoding: Leviathan *et al.* [1] introduced the canonical draft–verify scheme: a smaller *draft* model proposes k tokens autoregressively and the larger *target* verifies them in a single parallel forward pass, with modified rejection sampling guaranteeing that the output distribution is identical to that of the target. Chen *et al.* [2] concurrently proposed an equivalent scheme. The expected per-cycle latency is

$$L = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \qquad T_{\text{draft}} = k \cdot t_{\text{draft step}}, \qquad (1)$$

where $\tau \in [1, k+1]$ is the expected number of accepted tokens per cycle. Wall-clock speedup $S = L_{\text{target}}/L$ therefore depends on the draft/target cost ratio and the empirical token acceptance rate α. Stern *et al.* [3] earlier explored blockwise parallel decoding within a single model.

b) Reducing drafting cost: Medusa [4] eliminates the external draft by attaching multiple prediction heads to the target model and verifying the resulting candidate tree in one forward pass. The EAGLE family [5]–[7] keeps an external draft but conditions it on target hidden features, reporting substantial speedups under their settings. Despite these refinements, T_{draft} remains linear in k, forcing EAGLE-3 to a single transformer layer and capping reported production-framework serving throughput (e.g., in SGLang/vLLM) at about 2–3× on strong GPUs, well below the paper's idealized best-case decoding speedup of up to $6.5\times$.

c) Diffusion-based drafters: Recent work directly attacks the linear-in-k drafting bottleneck. DiffuSpec [8] and SpecDiff-2 [9] explore diffusion-based parallel drafting to improve speculative-decoding throughput. PARD [10] focuses on a low-cost parallel draft model adaptation. DFlash [11] uses a lightweight 5-layer block-diffusion drafter conditioned on target hidden features and reports up to $5\times$ speedup over autoregressive baselines in real-world serving scenarios on B200 hardware (over $6\times$ under idealized H200 decoding benchmarks).

d) Cascaded speculative control: Recent cascaded architectures dynamically balance inference cost and generation quality. CAS-Spec builds an on-the-fly hierarchy of draft stages from a single target model using Dynamically Switchable Inference Acceleration (DSIA) strategies (for example, layer sparsity and activation quantization), coordinated by a Dynamic Tree Cascade (DyTC) algorithm [12]. Speculative Cascades applies a deferral rule, invoking larger models only for hard inputs while routing easier cases to smaller models [13]. Although these methods report strong acceleration, they typically require target-side architectural changes, extra training, or complex serving stacks. By contrast, our adaptive-k policy framing is a training-free, architecture-agnostic runtime controller that adapts decisions online from acceptance signals, targeting runtime stability under consumer-GPU constraints rather than peak data-center throughput.

e) Precision and systems deployment: Deployment-oriented inference work shows that quantization and memory traffic are central to practical throughput gains on constrained hardware. Low-precision techniques such as LLM.int8() [14] reduce weight movement and can interact with speculative decoding in two opposing ways: lower per-step cost for draft/verify passes, but possible shifts in acceptance behavior when verifier and drafter precision settings differ. This interaction motivates reporting speed and quality jointly, rather than throughput alone.

f) Distributed and edge relevance: In distributed and mobile computing settings, local inference can reduce round-trip latency and cloud dependency for interactive workloads. In this context, speculative decoding is relevant beyond chatbot serving, including edge language interfaces where bounded latency and predictable runtime matter as much as peak throughput.

g) Position of this work: This work focuses on a controlled consumer-GPU evaluation of vanilla speculative decoding with same-family model pairing (Qwen3-4B target, 0.6B draft) under deterministic and stochastic regimes. Rather than competing with data-center throughput claims, we characterize practical operating points across desktop RTX4090 and RTX5090-laptop profiles, including acceptance dynamics, quality drift, and runtime-sensitive configuration choices. We then use these results to evaluate an adaptive-k run set as a runtime-stability extension.

III. Experimental Protocol

A. Datasets and sample sizes

We evaluate on three benchmarks spanning reasoning, knowledge, and summarization:

1) **GSM8K** [15] test subset: 300 samples.
2) **MMLU** [16] subset: 500 samples (5 subjects $\times$ 100 each).
3) **CNN/DailyMail** [17] subset: 200 samples.

Sampling policy: Primary benchmarking uses fixed random seed $= 42$, stratified sampling where applicable, and a frozen sample-ID manifest generated before experimentation. To assess run-to-run stability, we additionally repeated the Qwen3 RTX4090 k=4 settings (deterministic and stochastic) at seeds 123 and 999 on the same manifests. Each output CSV records seed and sample identifier, enabling paired analysis at sample and seed levels where repeats are available.

B. Prompt set

We use the following fixed prompt templates for each task:

GSM8K:

You are a careful math assistant. Solve the problem step by step and end with Final Answer: <number>.
Question: {question}

MMLU:

You are a knowledgeable assistant. Choose exactly one option.
Question: {question}
Options: A. {A} B. {B} C. {C} D. {D}
Answer:

CNN/DailyMail:

Summarize the following article in 3–4 sentences, preserving key facts and entities.
Article: {article}
Summary:

a) Objective and hypotheses: This study quantifies practical speculative-decoding gains on consumer hardware under fixed data and prompt controls. We test four hypotheses:

- **H1:** Same-family speculative decoding yields positive net speedup ($S > 1$) over autoregressive decoding.
- **H2:** Increasing draft length from k=4 to k=8 improves speedup, while k=16 may reduce gains due to additional drafting and verification overhead.
- **H3:** Increasing k tends to improve accepted block size but can reduce end-to-end speed once drafting/verification overhead dominates.
- **H4:** Deterministic decoding is more runtime-efficient and quality stable than stochastic decoding for the same draft and k.

TABLE I
Hardware profile for the desktop RTX4090 run family.

Component	Specification
GPU	NVIDIA RTX4090 GPU
GPU memory	24 GB GDDR6X
Memory bandwidth	1008 GB/s
CUDA cores	16384
Architecture	Ada Lovelace
Power profile	TGP class 450 W
Software stack	PyTorch + Transformers (pinned env)

TABLE II
Hardware profile for the RTX5090-laptop run family.

Component	Specification
GPU	NVIDIA GeForce RTX 5090 Laptop GPU
GPU memory	24 GB GDDR7
Memory bandwidth	896 GB/s
CUDA cores	10496
Architecture	NVIDIA Blackwell
Power profile	Laptop TGP class up to 150 W (OEM-dependent)
Software stack	PyTorch + Transformers (pinned env)

b) Hardware and software controls: We report two consumer-GPU run families: an NVIDIA RTX4090 desktop profile and an RTX5090-laptop profile. Both use the same Python runtime entry points, prompt templates, tokenizer settings, and serial execution policy to minimize cross-run contention. We keep software dependencies pinned within each run family and preserve frozen manifests for paired comparison.

c) Runtime setup: We keep fixed step shapes, preallocate static draft/verify buffers, and warm up the inference path before measurement. During decoding, each cycle updates buffer contents under shape guards and falls back to eager execution when required. This preserves output semantics and reduces avoidable runtime-path variance in short speculative cycles.

d) Model pairing: We evaluate two same-family pairings. For Qwen2.5, the target is Qwen2.5-3B-Instruct with Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct drafts on the RTX4090 run family. For Qwen3, the target is Qwen3-4B-Instruct with a Qwen3-0.6B-Instruct draft on RTX4090 and RTX5090-laptop run families. For each family, fixed-policy experiments sweep $k \in \{4, 8, 16\}$ under deterministic/stochastic regimes with regime-matched autoregressive baselines. All configurations process fixed manifests to preserve paired comparability.

e) Task-entropy interpretation: We treat the three tasks as distinct decoding-entropy regimes to interpret acceptance behavior: GSM8K is a lower-entropy chain reasoning with concentrated next-token mass; MMLU is medium-entropy constrained QA; CNN/DailyMail is higher-entropy open-ended summarization. This framing is used only for interpretation of acceptance and quality trends, not as an optimization objective.

f) Decoding regimes and grid: Two regimes are evaluated:

- **Deterministic:** greedy decoding (T=0.0, top-p=1.0; sampling disabled).
- **Stochastic:** sampling with T=0.7, top-p=0.9, and no top-k truncation.

For the primary fixed-policy matrix, we sweep $k \in \{4, 8, 16\}$ under both regimes (6 configurations), plus regime-matched autoregressive baselines.

g) Endpoints: The primary endpoint is paired wall-clock speedup $S = L_{\text{autoregressive}}/L_{\text{spec}}$ as reported by the experiment summary artifact for each draft–k–regime combination. Secondary endpoints include acceptance rate α, effective accepted block size B_{eff}, mean per-sample latency, throughput (tokens/s), task-level quality deltas, time to first token (TTFT), time per output token (TPOT), and 99th-percentile (P99) latency. Statistical reporting uses sample-level summaries (mean±std) and one-sided paired t-tests with Holm-Bonferroni correction and Cohen's d effect sizes, comparing each fixed-k configuration against its regime-matched autoregressive baseline. For repeated Qwen3 RTX4090 k=4 settings, we also report seed-to-seed stability summaries across seeds $\{42, 123, 999\}$.

IV. Evaluation metrics

Table III defines the exact reporting metrics used in this study.

A. Success criteria

Primary:

1) Mean speedup $S \geq 1.3\times$ with Holm-Bonferroni-corrected $p_S < 0.05$.
2) Absolute quality drop $\leq$ 1.0 point on GSM8K and MMLU in deterministic regime.

Secondary: Identify the best (k, regime) maximizing S under quality constraints, report sample-level dispersion (mean±std), report seed-level stability where repeated seeds are available, and test whether ranking trends transfer across hardware profiles even when absolute speedups differ.

V. Implementation and Reproducibility

a) Code organization: The experiment pipeline is implemented under the project source directory with modular components for configuration, data loading, and autoregressive baselines, speculative decoding, metric aggregation, and runtime orchestration. The notebook driver executes these modules in fixed phase order and writes artifacts to the results directory.

b) Model loading and precision: In the current configuration, target and draft models are loaded in 16-bit floating point (FP16) for the main sweep, with quantization controls retained in configuration for alternative runs. The tokenizer and generation limits are kept constant between baselines and speculative runs to preserve paired comparability.

c) Baselines: We run one autoregressive baseline per regime (deterministic and stochastic) with the same manifests and output-length constraints as speculative runs. Each baseline artifact stores per-sample latency, output length, and task predictions.

d) Speculative decoding implementation.: The speculative loop follows standard token-level draft–verify logic. At each cycle, the draft proposes up to k tokens autoregressively; the target verifies the proposal in one pass over the extended prefix; the accepted prefix is committed up to the first mismatch; and one correction token is added from the target distribution when needed. This procedure preserves target-level generation semantics while exposing acceptance statistics needed for analysis. Both draft and target states are tracked with key-value (KV) caches throughout the decode loop. Figure 1 summarizes this implementation flow.

e) CUDA-graph runtime note: In the Qwen3 run families, graph-capture metadata differs by hardware. The RTX4090 fixed-policy matrix reports stable captured execution, whereas RTX5090-laptop sensitivity runs show that graph-enabled toggles do not consistently improve end-to-end latency. We therefore treat CUDA-graph usage as an implementation detail rather than a standalone performance claim, and base conclusions on end-to-end fixed-policy speedup trends.

f) Dynamic-scheduling bottleneck in native PyTorch paths: Fallback eager execution should not be viewed only as an implementation miss. It reflects a broader systems challenge for end-to-end dynamic speculation. When acceptance-dependent control flow, sampling branches, and cache updates change per step, preserving lossless semantics often requires CPU-visible orchestration between kernels. Without a deeper runtime/kernel redesign, this serial control overhead can erase expected gains from speculative parallelism, as also emphasized in modern inference runtime roadmaps for dynamic scheduling.

g) Tokenizer isomorphism as an enabling design choice: Same-family draft/target pairing is a central engineering decision in this study. Because the draft and target tokenizers are isomorphic, speculative verification avoids per-step vocabulary alignment and token translation. Cross-tokenizer pipelines introduce additional CPU-side mapping overhead that can materially reduce or negate speculative speedup on constrained hardware.

h) Adaptive-k implementation note: For adaptive-k runs, we reuse the same draft–verify core and switch only the policy controller: k is adapted online from recent acceptance behavior with guarded fallback to stable paths when acceptance collapses. We treat the adaptive-k run set as a runtime policy layer over the fixed-policy implementation, not as a separate decoding algorithm.

i) Regimes and outputs: The same verification core is used for deterministic and stochastic modes, with only the sampling policy changed by regime parameters. Each configuration writes a dedicated CSV artifact with per-sample runtime, token counts, and quality-relevant outputs.

TABLE III
METRIC DEFINITIONS USED IN THIS STUDY.

Metric	Symbol	Computation	Unit	Direction
End-to-end latency	T	completion_time − request_start	s	Lower
Throughput	R_{tok}	output_tokens / T	tokens/s	Higher
Time-to-first-token	TTFT	first_token_time − request_start	ms	Lower
Time-per-output-token	TPOT	(completion_time − first_token_time) / output_tokens	ms/token	Lower
Acceptance rate	α	accepted_draft_tokens / proposed_draft_tokens	ratio	Higher
Effective block size	B_{eff}	accepted_draft_tokens / verify_steps	tokens/step	Higher
Relative speedup	S	baseline_latency / speculative_latency	$\times$	Higher
GSM8K quality	Q_{gsm}	correct / total	%	Higher
MMLU quality	Q_{mmlu}	correct / total	%	Higher
CNN/DailyMail ROUGE-L	Q_{sum}	standard ROUGE-L F1 (rouge-score, stemmed)	%	Higher
CNN/DailyMail BLEU	Q_{bleu}	sentence-level BLEU (NLTK, method1 smoothing)	%	Higher
CNN/DailyMail BERTScore	Q_{bert}	BERTScore F1 (DistilBERT-base-uncased)	score	Higher
Quality delta	ΔQ	$Q_{\text{spec}} - Q_{\text{base}}$	points	≈ 0 or $+$
Speedup significance	p_S	one-sided paired t-test for $H_0 : S \leq 1$, Holm-Bonferroni corrected	p-value	Lower
Speedup effect size	d_S	Cohen's d for paired baseline/speculative latency	σ units	Higher
Speedup dispersion	σ_S	std(S over evaluated samples within one seed)	$\times$	Lower
Seed-level stability	σ_{seed}	std(metric mean across repeated seeds for the same config)	metric unit	Lower

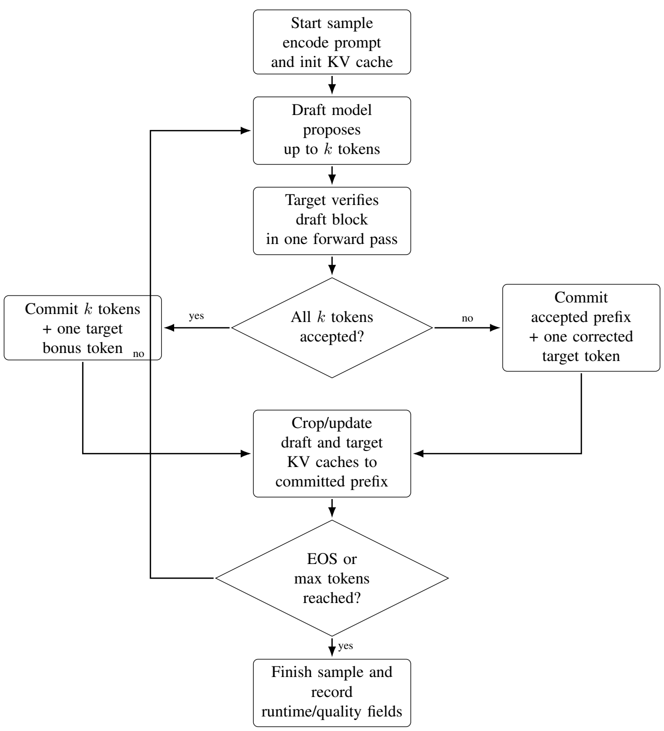

Fig. 1. Vanilla speculative decoding implementation flow. The target model remains the verifier of record; accepted draft prefixes are committed, and caches are cropped to preserve exact target-consistent decoding.

VI. RESULTS

This section reports fixed-policy results for Qwen2.5 and Qwen3 families across two consumer-GPU setups: desktop RTX4090 and RTX5090-laptop. We focus on paired speedup S, acceptance dynamics (α, B_{eff}), and token-timing metrics (time to first token, TTFT; time per output token, TPOT), then summarize portability implications.

A. RQ1: Does speculative decoding accelerate inference?

On RTX4090, Qwen2.5 and Qwen3 both show clear acceleration in most fixed-policy settings. For Qwen2.5 (Table IV), all six reported points exceed baseline, with best speedup at 0.5B, k=4, deterministic (S=3.0674). For Qwen3 (Table V), five of six points exceed baseline, with best speedup at 0.6B, k=4, deterministic (S=2.8548). The one below-baseline case is Qwen3 high-k stochastic (k=16, S=0.9719), indicating that large proposal length can remove net latency gains.

B. RQ2: How do k and acceptance affect net latency?

Both families retain the same systems trend: larger k increases accepted block size but degrades end-to-end speed. On RTX4090, Qwen2.5 mean speedup by k is $\bar{S}_{k=4} = 2.9164$, $\bar{S}_{k=8} = 2.4303$, $\bar{S}_{k=16} = 1.8118$, while Qwen3 shows $\bar{S}_{k=4} = 2.6673$, $\bar{S}_{k=8} = 1.8258$, $\bar{S}_{k=16} = 1.0405$. In both tables, B_{eff} rises with k while speedup falls, indicating an overhead inflection where additional drafting and verification cost dominates block-size benefits.

C. RQ3: Deterministic versus stochastic regime

For Qwen3, deterministic decoding is faster than stochastic at every matched k: $\Delta S_{k=4}$=0.3750, $\Delta S_{k=8}$=0.2779, $\Delta S_{k=16}$=0.1372 (deterministic minus stochastic), with mean gap 0.2633 speedup units (Table V). For Qwen2.5, deterministic is also faster at k=4 and k=8 (Table IV), with a small reversal at k=16. Across both families, deterministic generally provides a stronger runtime profile at low-to-mid k.

D. RQ4: Cross-hardware portability (RTX4090 vs RTX5090-laptop)

The ordering pattern transfers across devices (low-k best, deterministic preferred, and monotonic decline with larger k), but absolute gains do not. Under the tested RTX5090-laptop software/runtime stack, all reported fixed-policy points are slower than autoregressive baseline. This indicates that conclusions are more portable than absolute speedup magnitudes.

TABLE IV
QWEN2.5 FIXED-POLICY RESULTS ON RTX4090 (TARGET: 3B, DRAFT: 0.5B) AGAINST REGIME-MATCHED AUTOREGRESSIVE BASELINES. SPEEDUP IS COMPUTED AS A RATIO OF MEANS AGAINST THE CORRESPONDING BASELINE.

Config	S	α	B_{eff}	T_{mean} (s)	TTFT (ms)	TPOT (ms)
0.5B, k=4, det	**3.0674**	0.4212	1.66	3.7210	102.35	29.06
0.5B, k=4, stoch	2.9394	0.4080	1.61	3.9631	105.26	30.89
0.5B, k=8, det	2.4542	0.3515	2.72	4.6508	162.76	36.73
0.5B, k=8, stoch	2.4063	0.3444	2.67	4.8411	165.95	38.04
0.5B, k=16, det	1.8050	0.2539	3.76	6.3234	277.43	49.34
0.5B, k=16, stoch	1.8185	0.2516	3.73	6.4059	278.85	49.81

TABLE V
QWEN3 FIXED-POLICY RESULTS ON RTX4090 AGAINST REGIME-MATCHED AUTOREGRESSIVE BASELINES. SPEEDUP IS COMPUTED AS A RATIO OF MEANS AGAINST THE CORRESPONDING BASELINE.

Config	S	α	B_{eff}	T_{mean} (s)	TTFT (ms)	TPOT (ms)
0.6B, k=4, det	**2.8548**	0.3362	1.33	4.8881	132.57	41.23
0.6B, k=4, stoch	2.4798	0.2672	1.06	5.6196	134.54	45.51
0.6B, k=8, det	1.9648	0.2378	1.86	7.1024	204.99	58.76
0.6B, k=8, stoch	1.6869	0.1916	1.50	8.2612	207.82	64.67
0.6B, k=16, det	1.1091	0.1292	1.96	12.5817	349.58	94.24
0.6B, k=16, stoch	0.9719	0.1082	1.64	14.3390	354.82	106.03

TABLE VI
QWEN3 FIXED-POLICY RESULTS ON RTX5090-LAPTOP. ORDERING TRENDS ARE PRESERVED (BEST AT LOW k, DETERMINISTIC > STOCHASTIC), BUT ABSOLUTE SPEEDUPS ARE BELOW 1 UNDER THE TESTED RUNTIME STACK.

Config	S	α	B_{eff}
0.6B, k=4, det	0.6743	0.3353	1.32
0.6B, k=4, stoch	0.5924	0.2845	1.06
0.6B, k=8, det	0.4613	0.2538	1.85
0.6B, k=8, stoch	0.3995	0.2273	1.50
0.6B, k=16, det	0.2588	0.1803	1.96
0.6B, k=16, stoch	0.2222	0.1543	1.64

E. Sample-level and seed-level stability, TTFT/TPOT, and statistical testing detail

To provide stronger statistical detail, we report sample-level dispersion (mean±std across the n=1000 evaluated samples per configuration) alongside the ratio-of-means speedup in Table V. Per-sample deterministic speedup is 2.7561 ± 0.5950 (k=4), 2.0010 ± 0.6147 (k=8), and 1.3549 ± 0.6533 (k=16); per-sample stochastic speedup is 2.4945 ± 0.4565, 1.8303 ± 0.5644, and 1.2173 ± 0.6206, respectively. TTFT rises with k in both regimes (deterministic: 132.57 ± 73.85 to 349.58 ± 72.72 ms from k=4 to k=16; stochastic: 134.54 ± 73.51 to 354.82 ± 72.85 ms), matching Table V exactly, while TPOT widens correspondingly (deterministic: 41.23 ± 9.59 to 94.24 ± 39.46 ms; stochastic: 45.51 ± 8.93 to 106.03 ± 41.23 ms), consistent with the overhead interpretation. This spread reflects genuine per-prompt variability (task mix and output-length differences) within each run seed.

We additionally ran a seed-stability extension for Qwen3 RTX4090 at k=4 using seeds 123 and 999 (same manifests, n=1000 each). For deterministic decoding, mean latency is 5.2244 ± 4.0698 s (seed 123) and 5.2331 ± 4.0705 s (seed 999), with TTFT 141.14 ± 74.33 ms and 141.56 ± 74.90 ms, TPOT 44.05 ± 10.23 ms and 44.16 ± 10.27 ms, acceptance α=0.3354 in both seeds, and B_{eff}=1.31 in both seeds. For stochastic decoding, mean latency is 6.0610 ± 4.7974 s (seed 123) and 5.7658 ± 4.5195 s (seed 999), with TTFT 143.66 ± 74.88 ms and 143.79 ± 74.89 ms, TPOT 50.02 ± 11.00 ms and 47.87 ± 9.76 ms, acceptance α=0.2750 and 0.2942, and B_{eff}=1.07 and 1.15. Relative to the seed-42 row in Table V, these repeats preserve regime ordering and show modest seed sensitivity at this operating point.

We further report one-sided paired t-tests (H_0: $S \leq 1$) comparing each fixed-k configuration against its regime-matched autoregressive baseline, paired by sample, with Holm-Bonferroni correction across all six configurations and Cohen's d effect sizes. Five of six configurations show highly significant, large-effect speedup after correction: deterministic k=4 (d=1.18, $p_{\text{adj}} < 10^{-190}$), k=8 (d=1.10, $p_{\text{adj}} < 10^{-172}$), k=16 (d=0.29, $p_{\text{adj}} < 10^{-18}$), and stochastic k=4 (d=1.24, $p_{\text{adj}} < 10^{-202}$), k=8 (d=1.13, $p_{\text{adj}} < 10^{-179}$). Stochastic k=16 is the exception: the corrected test is not significant ($d = -0.08$, $p_{\text{adj}} = 0.996$), consistent with its below-baseline mean speedup (S=0.9719) reported above. These tests use the complete seed-42 full-grid matrix; the added seed-123/999 runs are currently available for the k=4 Qwen3 stability extension only.

For CNN/DailyMail (deterministic, n=200), quality also shifts under speculative decoding across all three reported metrics relative to the autoregressive baseline ($Q_{\text{sum}}=19.58 \pm 5.61$, $Q_{\text{bleu}}=3.20 \pm 3.09$, $Q_{\text{bert}}=0.793 \pm 0.031$): k=4 gives $Q_{\text{sum}}=11.50 \pm 4.85$, $Q_{\text{bleu}}=1.39 \pm 1.81$, $Q_{\text{bert}}=0.710 \pm 0.039$; k=8 gives 14.04 ± 4.93, 1.66 ± 1.98, 0.729 ± 0.038; and k=16 gives 14.92 ± 5.14, 2.04 ± 2.36, 0.741 ± 0.037.

F. Cross-family consistency (Qwen2.5 vs Qwen3)

To address model-family applicability, we compare Qwen2.5 RTX4090 results with the Qwen3 results. The absolute speedup scale differs across runs, but the core qualitative patterns are consistent across both families on RTX4090: (i) best operating point at low k (especially k=4), (ii) worsening speedup as k increases, and (iii) deterministic preference in most matched settings.

Concretely, the Qwen2.5 table reports best fixed speedup at 0.5B, k=4, deterministic (S=3.0674), with mean-k trend $\bar{S}_{k=4} = 2.9164 > \bar{S}_{k=8} = 2.4303 > \bar{S}_{k=16} = 1.8118$. In the Qwen3 RTX4090 run family, we observe the same ordering with different magnitudes ($\bar{S}_{k=4} = 2.6673 > \bar{S}_{k=8} = 1.8258 > \bar{S}_{k=16} = 1.0405$).

These aligned trend-level findings support a bounded claim: operating-policy ranking appears more portable across these two model families than absolute throughput values, which remain implementation- and hardware-path dependent.

VII. Discussion

A. Main empirical takeaways

The Qwen2.5 and Qwen3 evidence supports three robust observations. First, on RTX4090, speculative decoding can provide substantial acceleration, but not for every setting; high-k stochastic can drop below baseline. Second, speedup declines as k increases, even when accepted block size grows. Third, deterministic decoding consistently outperforms stochastic decoding for matched k in both run families.

The cross-hardware extension adds an important qualification. While policy ordering trends transfer from RTX4090 to RTX5090-laptop, absolute speedup magnitudes do not. This supports a practical interpretation: generalization claims should emphasize portable trends and decision rules, not fixed throughput numbers. The adaptive-k run set was evaluated at a single fixed rescue-threshold, window, and cooling configuration in this study, reinforcing its framing as a stability-oriented mechanism rather than a tuned throughput optimizer; a systematic sweep over these hyperparameters is identified as future work in Section VIII.

B. Novelty and contribution scope

This paper's contribution is primarily empirical and systems-oriented. We do not claim a new speculative decoding algorithm that universally improves throughput. Instead, we provide a controlled, reproducible characterization that links acceptance dynamics, token timing, and latency tails to practical operating choices on consumer hardware.

C. Variance and evaluation depth

To strengthen rigor, we report sample-level mean±std summaries, time to first token (TTFT), time per output token (TPOT), and Holm-Bonferroni-corrected paired t-tests with Cohen's d effect sizes. These additions improve variability reporting and statistical clarity, and show that five of six fixed-k configurations achieve significant, large-effect speedup, while stochastic k=16 does not. We further add a seed-stability extension for Qwen3 RTX4090 at k=4 (seeds 123 and 999 in addition to seed 42), which preserves deterministic-over-stochastic ordering and shows small seed-to-seed drift in acceptance and token timing at this operating point. We also expand quality reporting beyond a single summarization metric (ROUGE-L, BLEU, and BERTScore, computed on the real CNN/DailyMail outputs). All three metrics move together and show a genuine quality cost under speculative decoding relative to the autoregressive baseline, largest at k=4 and partially recovering at higher k – the reverse of the speed trend, indicating a real speed–quality trade-off rather than a free lunch at low k. We acknowledge that none of these metrics fully capture factuality or task-specific utility.

VIII. Threats to Validity

Hardware and runtime dependence. Results remain sensitive to implementation stack and runtime path. The RTX4090 and RTX5090-laptop comparison demonstrates that ranking trends can transfer while absolute gains diverge.

Model-family scope. This study centers on same-family Qwen2.5 and Qwen3 target/draft pairings. Cross-family speculative pairings may show different acceptance and quality behavior.

Evaluation breadth. The benchmark suite covers reasoning, knowledge QA, and summarization, but does not exhaust long-context, safety, or factual consistency requirements.

Sample-level versus seed-level dispersion. The mean±std figures reported in Section VI reflect per-sample dispersion within each evaluation seed. We now include repeated seed runs (123 and 999) for the Qwen3 RTX4090 k=4 deterministic and stochastic settings, providing initial seed-level stability evidence. However, full multi-seed repetition across the entire $k \in \{4, 8, 16\}$ grid and all model/hardware families was not performed and remains future work.

Adaptive-k hyperparameter scope. The adaptive-k run set was evaluated at a single fixed rescue-threshold, window-size, and cooling-period configuration. A systematic ablation across these hyperparameters was not performed and is left for future work.

IX. Conclusion

This paper presents reproducible deployment guidance for consumer GPUs using same-family speculative decoding in both Qwen2.5 and Qwen3 settings. On RTX4090, deterministic k=4 remains the strongest operating point family-wise, with monotonic degradation as k increases. Cross-hardware validation on RTX5090-laptop preserves the same policy ordering trend but not the same absolute speedups, emphasizing that portability must be interpreted at the trend level.

The adaptive-k run set remains useful as a runtime-stability policy layer, but is not presented as a universal throughput optimizer. Future work will expand hardware diversity, include additional model families, and evaluate longer-context and richer quality diagnostics.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2211.17192

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," in *arXiv preprint arXiv:2302.01318*, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2302.01318

[3] M. Stern, N. Shazeer, and J. Uszkoreit, "Blockwise parallel decoding for deep autoregressive models," *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. [Online]. Available: https://doi.org/10.48550/arXiv.1811.03115

[4] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," *arXiv preprint arXiv:2401.10774*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2401.10774

[5] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," *arXiv preprint arXiv:2401.15077*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2401.15077

[6] ——, "EAGLE-2: Faster inference of language models with dynamic draft trees," *arXiv preprint arXiv:2406.16858*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2406.16858

[7] ——, "EAGLE-3: Scaling up inference acceleration of large language models via training-time test," *arXiv preprint arXiv:2503.01840*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2503.01840

[8] G. Li, Z. Fu, M. Fang, Q. Zhao, M. Tang, C. Yuan, and J. Wang, "DiffuSpec: Unlocking diffusion language models for speculative decoding," *arXiv preprint arXiv:2510.02358v1*, 2025, https://arxiv.org/abs/2510.02358v1. [Online]. Available: https://doi.org/10.48550/arXiv.2510.02358

[9] J. Sandler, J. K. Christopher, T. Hartvigsen, and F. Fioretto, "SpecDiff-2: Scaling diffusion drafter alignment for faster speculative decoding," *arXiv preprint arXiv:2511.00606*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2511.00606

[10] Z. An, H. Bai, Z. Liu, D. Li, and E. Barsoum, "PARD: Accelerating llm inference with low-cost parallel draft model adaptation," *arXiv preprint arXiv:2504.18583*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2504.18583

[11] J. Chen, Y. Liang, and Z. Liu, "DFlash: Block diffusion for flash speculative decoding," *arXiv preprint arXiv:2602.06036*, 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2602.06036

[12] Z. Ning, J. Shao, R. Xu, X. Guo, J. Zhang, C. Zhang, and X. Li, "CAS-Spec: Cascade adaptive self-speculative decoding for on-the-fly lossless inference acceleration of llms," *arXiv preprint arXiv:2510.26843*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2510.26843

[13] H. Narasimhan, W. Jitkrittum, A. S. Rawat, S. Kim, N. Gupta, A. K. Menon, and S. Kumar, "Faster cascades via speculative decoding," in *Proceedings of the IEEE Conference (arXiv preprint)*, 2024. [Online]. Available: https://arxiv.org/abs/2405.19261

[14] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2208.07339

[15] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2110.14168

[16] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," *arXiv preprint arXiv:2009.03300*, 2020. [Online]. Available: https://doi.org/10.48550/arXiv.2009.03300

[17] A. See, P. J. Liu, and C. D. Manning, "Get to the point: Summarization with pointer-generator networks," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017. [Online]. Available: https://doi.org/10.48550/arXiv.1704.04368